

Missile Trajectory Optimization Using Ant Colony Algorithm

Optimization for Artificial Intelligence final project presentation

Project Idea

- The primary goal of this project is to automate the design of a high-power rocket to precisely match a specific **flight trajectory**.
- Rather than relying on manual iteration, advanced optimization algorithms are employed to *reverse engineer* the optimal **geometry** and **propulsion** parameters.
- Two distinct approaches are implemented and compared: a **Continuous Ant Colony Optimization Algorithm (ACO-R)** and a standard **Binary Genetic Algorithm (Binary GA)**.
- By interfacing these algorithms with a realistic physics simulator, the study seeks to determine which method yields higher speed and accuracy in identifying the optimal missile design.

Outline

- ❖ Introduction & Task Settings
- ❖ Rocket Factory & Simulation
- ❖ The Ground Truth Rocket
- ❖ The Search Space: Parameters
- ❖ Fitness Functions
- ❖ Optimization: Ant colony algorithm for continuous domains (ACO-R).

- ❖ Implementation of Nelder-Mead Local search
- ❖ Optimization: Binary Genetic Algorithm (GA).
- ❖ Experimental Results: Single Trajectory Optimization
- ❖ Experimental Results: Triple Trajectory Optimization
- ❖ Experimental Results: Final Comparison
- ❖ Conclusions & Future Work

The background is a dark blue gradient filled with numerous small white dots representing stars. Three bright red streaks, resembling meteors or comets, are positioned diagonally across the frame. One streak is in the upper left, another in the lower left, and a third in the lower right. The text 'Introduction & Task Settings' is centered in the middle of the image in a white, bold, sans-serif font.

Introduction & Task Settings

The Inverse Design

- In traditional aerospace design, a missile is defined and its flight is simulated to see where it will land.
- But what happens when the problem is turned upside down?
- This project tackles the challenge of **Reverse Engineering Design**: starting from a predetermined ideal trajectory generated by a **Ground Truth rocket**, is it possible to use optimization algorithms to reconstruct a feasible geometry and propulsion needed to achieve it?
- To meet this objective, the implemented algorithms are tasked with identifying an optimal set of **design parameters** that yields flight performance metrics virtually identical to those of the reference **Ground Truth** rocket.

The Optimization Challenge

- Reconstruct the flight trajectory by tuning **11 continuous design parameters** covering the rocket's geometry, aerodynamics, and propulsion. The algorithms operate within strict, physically feasible ranges to generate candidate designs.
- Each candidate is tested via a physics simulation, comparing its performance against the reference **Golden Rocket**. The algorithms aim to minimize a **fitness function** derived from the error margins of four key flight metrics: **Range** (Distance), **Apogee** (Max Altitude), **Time of Flight** (TOF), **Time to Burnout** (TBO).

The Experimental Scenario

The optimization study is organized into two distinct phases to evaluate algorithmic adaptability:

- **Single Trajectory Optimization**, focuses on precision matching. The objective is to identify the configuration required to perfectly replicate a specific reference flight profile launched at an **85° inclination**. This serves as a baseline test for convergence accuracy in a static scenario.
- **Triple Trajectory Optimization**. In this multi-scenario context, a **single rocket design** must be optimized to meet performance targets across three different launch angles (**85°, 80°, and 75°**) simultaneously. This constraint forces the algorithm to identify a robust physical compromise rather than overfitting to a single condition.



Rocket Factory and Simulation

The Simulation Bridge: Parametric Generation

- The core of the simulation environment is the **Rocket Factory**, a procedural generation module acting as a bridge between the optimization algorithms and the physics engine.
- The class instantiates a **parametric rocket model** in real-time. An **11-dimensional** vector, representing the **design variables**, is mapped into a complete physical object defined by mass distribution, aerodynamic surfaces, and propulsion characteristics.
- This allows the optimizer to explore a continuous design space rather than a discrete catalog of parts.

Modelling Dynamics: Mass and Inertia

- To ensure the simulation fidelity, the mass properties are not approximated as a point mass but calculated analytically. The rocket is decomposed into structural primitives (nose cone, fuselage, fins, motor, and ballast).
- The **Inertia Tensor** is derived using the **Parallel Axis Theorem**, aggregating the local moments of inertia of each component relative to the global Center of Gravity (**CGCG**).
- This approach ensures that geometric changes, such as lengthening the body or enlarging fins, correctly impact the vehicle's rotational dynamics and stability in response to wind bursts.

Aerodynamic Constraints: Dynamic Drag

- A critical aspect of the model is the estimation of aerodynamic drag, specifically **Skin Friction Drag**, which depends on the vehicle's wetted surface area.
- The simulation dynamically adjusts the drag coefficient based on the optimized geometry (Radius and Length). Consequently, designs that attempt to maximize stability by excessively increasing the fuselage length incur a performance penalty due to increased friction.
- This constraint forces the optimizer to find a realistic compromise between aerodynamic stability and flight efficiency.

Synthetic Propulsion and Ground Truth

- Propulsion is modeled via **Synthetic Thrust Curve Generation**. Rather than using static motor files, a procedural algorithm generates a trapezoidal thrust profile constrained by the specific **Total Impulse** and **Burn Time** selected by the optimizer.
- Before the optimization begins, a reference vehicle, the **Golden Rocket** is instantiated with a specific set of valid parameters. This reference model is simulated to establish the **Ground Truth** performance metrics (Range, Apogee, Time of Flight) that the algorithms must strive to replicate.

The background of the image is a dark blue, starry night sky. Three bright red streaks, resembling meteors or rocket trails, are visible. One streak is in the upper left, another is in the lower left, and a third is in the lower right. The text "The Ground Truth Rocket" is centered in the middle of the image.

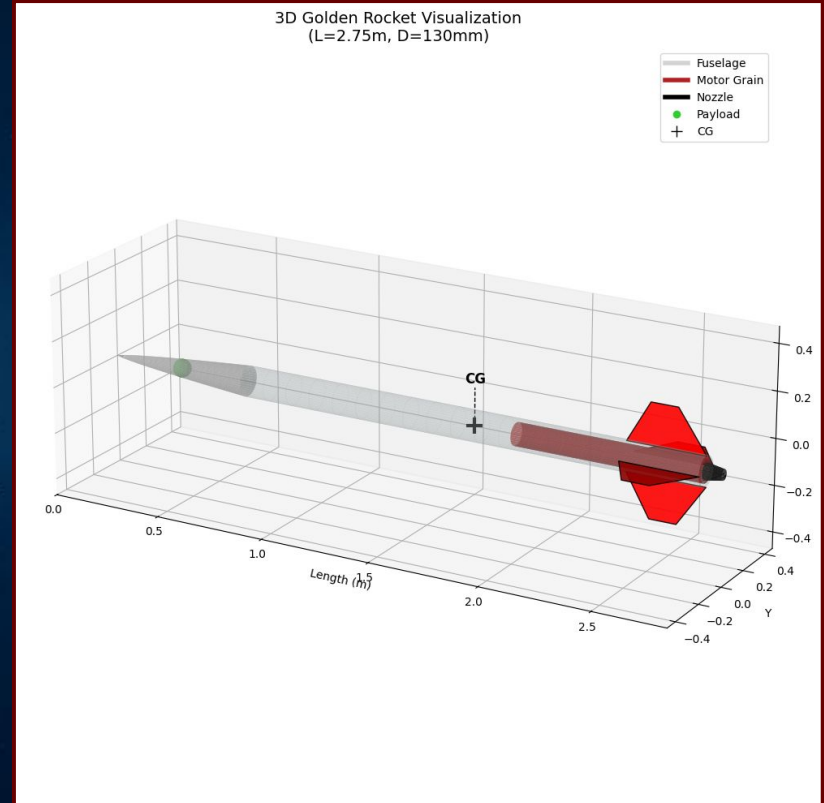
The Ground Truth Rocket

The Ground Truth Rocket: 3D visualization

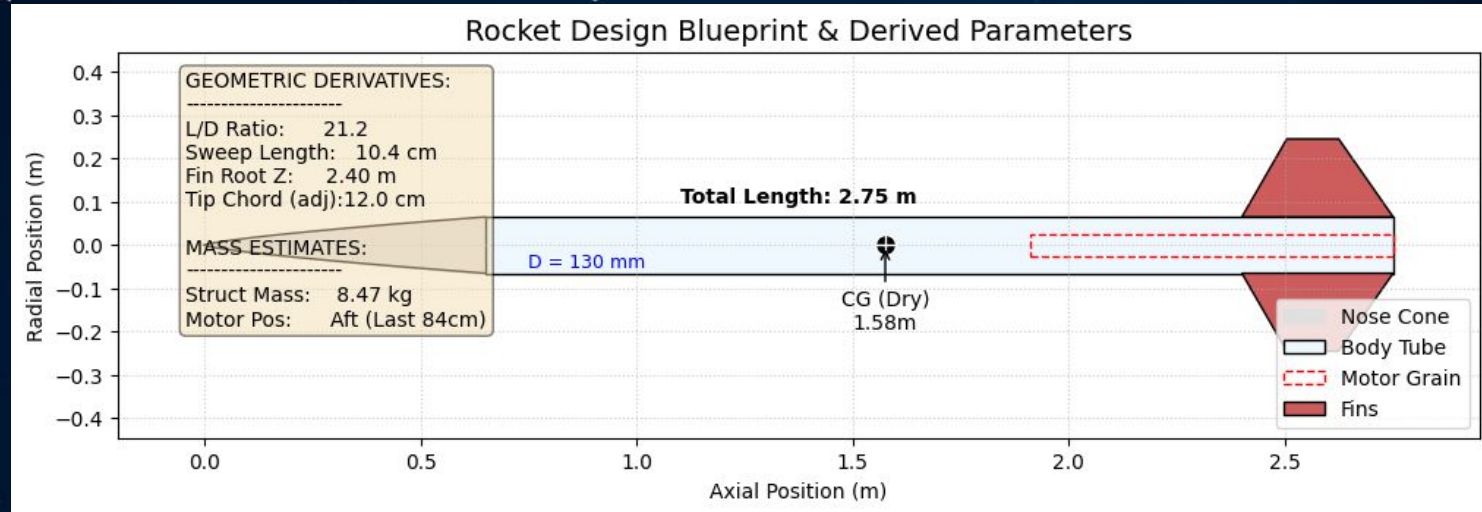
The process begins with the definition of a **Ground Truth** vehicle to establish a valid flight reference.

This model represents a realistic **Class-L high-power rocket**, configured to ensure structural feasibility and aerodynamic stability (*Static Margin* > 1.5 cal).

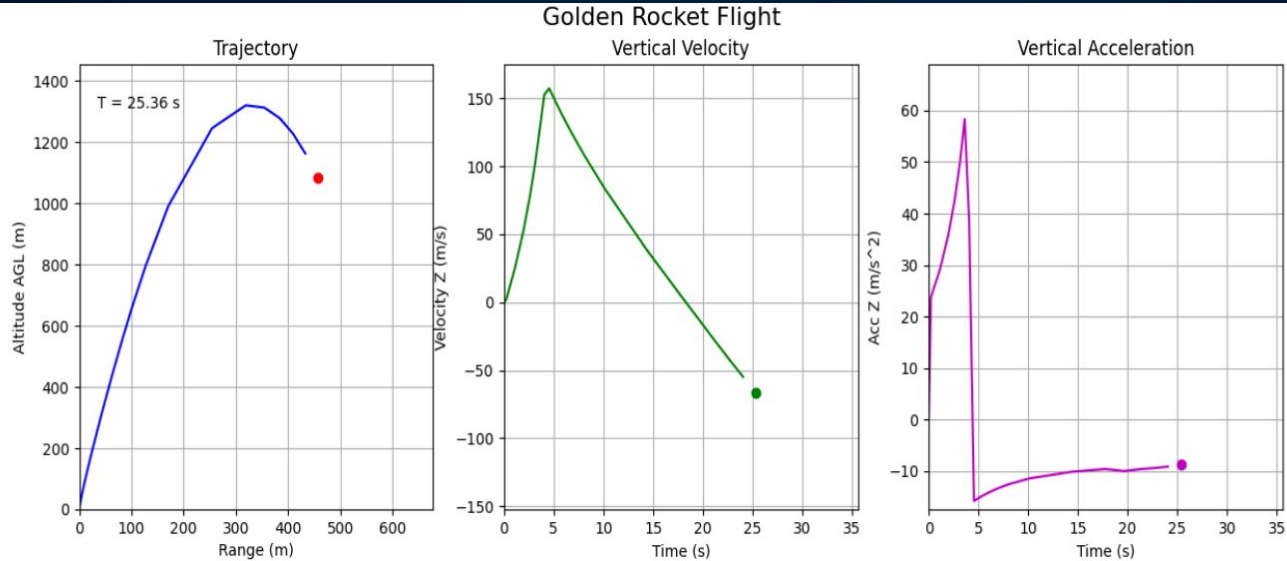
The trajectory metrics derived from this simulation serve as the precise targets for the optimization algorithms.



The Ground Truth Rocket: Blueprint and Derived Parameters



The Ground Truth Rocket: Flight Simulation





The Search Space: Parameters

The Search Space: Geometric Optimization Variables

Parameter	Symbol	Unit	Definition	Ground Truth target
Fuselage Radius	r	m	External radius of the main body tube.	0.065 m
Body Length	L_{body}	m	Length of the main fuselage tube (excluding nose)	2.10 m
Nose Length	L_{nose}	m	Length of the aerodynamic nose cone. Influences wave drag and CP position.	0.65m
Fin Span	S_{fin}	m	Height of the fins from the fuselage root to tip. Critical for lift and stability.	0.18m
Fin Root Chord	C_{root}	m	Length of the fin attached to the fuselage. Structural base of the fin.	0.35m
Fin Tip Chord	C_{tip}	m	Length of the fin at the outer tip. Determines fin area and shape.	0.12m
Fin Sweep Angle	Λ	deg	Angle of the fin's leading edge. Reduces drag at high speeds.	30.0°
Fin Position	X_{fin}	m	Distance from the rocket base. Fins at the rear maximize stability.	0.00m

The Search Space: Propulsion and Launch Constraints

Parameter	Symbol	Unit	Definition	Ground Truth target
Total Impulse	I_{tot}	Ns	Total energy of the motor. Determines the max altitude capability.	3200.00Ns
Burn Time	t_{burn}	s	Duration of thrust generation. Trade-off between high acceleration (short burn) and low drag losses (long burn).	4.5s
Launch Angle	θ	deg	Inclination of the launch rail relative to the ground (90° = Vertical).	85°

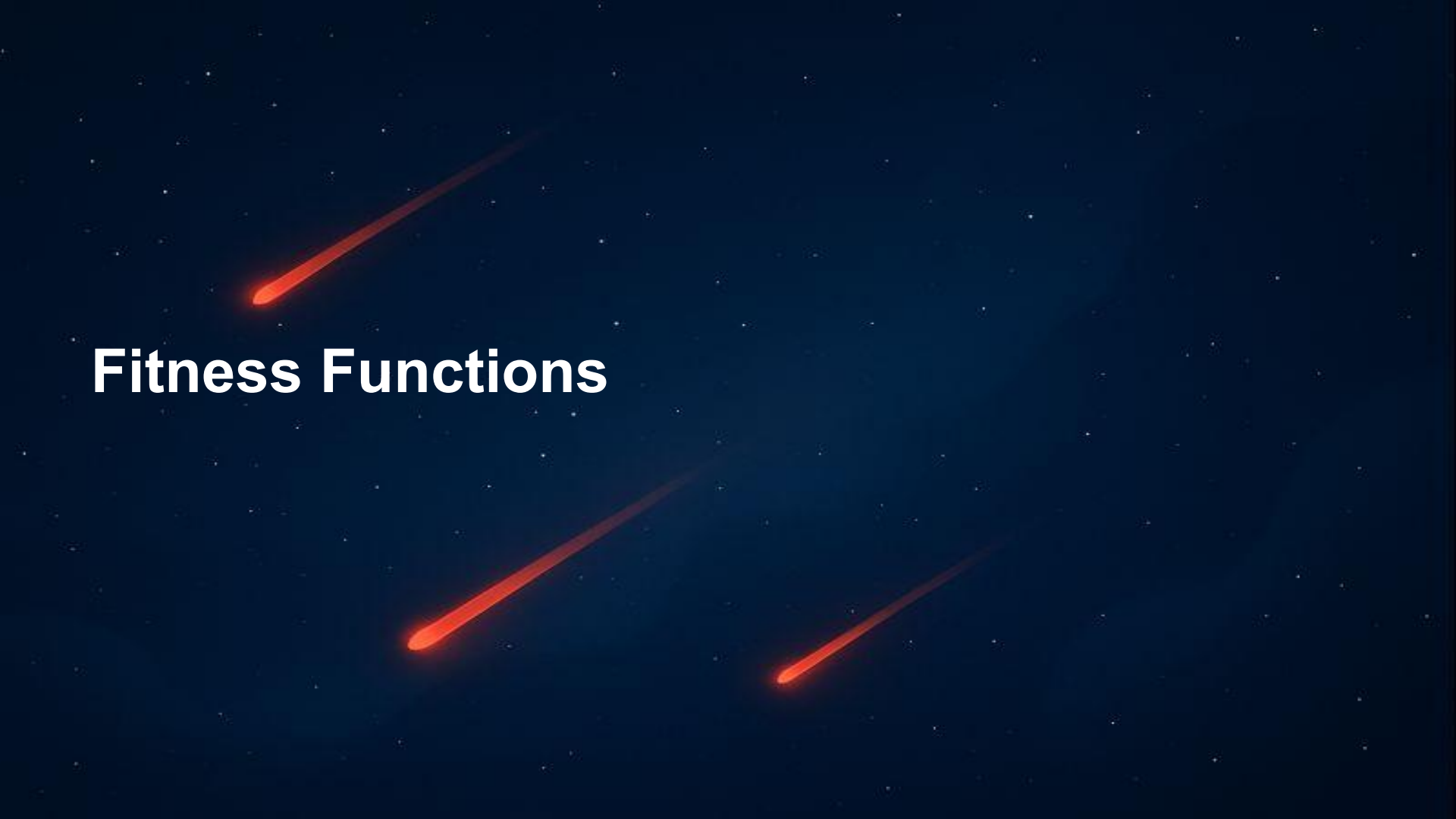
- **Burn Time** defines the input energy release rate but is also explicitly tracked in the cost function. This strict constraint forces the optimizer to identify the exact propulsion system characteristics.
- **Launch Angle θ** Shifts from a Degree of Freedom in the single-trajectory task to a Environmental Parameter in the multi-trajectory task.

Physical Design Constraints

- **Fuselage Radius (60–100 mm):** Enforces a minimum diameter to ensure internal volume for standard 98mm motor casings, preventing the algorithm from creating unrealistically thin darts to cheat drag.
- **Body Length (1.2–2.4 m):** Upper bounds prevent structural instability (body flex/flutter) and ensure transportability, while lower bounds guarantee sufficient stability margins (L/D ratio).
- **Impulse (2000–5000 Ns):** Constrains the search to **Class K and L** high-power motors.
- **Burn Time (2.0–6.0 s):** Prevents the generation of *explosive* motors (infinite thrust, zero time) or unrealistic *moon-burners*, keeping the thrust curve within chemical propellant limits.
- **Launch Angle (75°–88°):** Hard limits set to ensure range safety and avoid mathematical singularities at 90° (vertical).

Key Takeaway:

By tightening these bounds, the **ACO-R** and **GA** algorithms are forced to solve a **valid engineering problem** rather than exploiting mathematical loopholes in the physics engine.



Fitness Functions

Fitness Function for Single Trajectory Optimization

- The optimization problem is formulated as the **minimization** of a scalar cost function, which quantifies the deviation between the simulated flight dynamics and the reference ground truth data.

Parameter	Value	Unit	Description
Target Range	615.6	m	Euclidean distance from pad
Target Apogee	1322.3	m	Altitude AGL
Target TOF	35.60	s	Total flight duration
Target TBO	4.50	s	Time to Burnout

$$Fitness = \frac{RangeError}{10} + \frac{ApogeeError}{10} + \frac{TOFError}{1} + \frac{TBOError}{1}$$

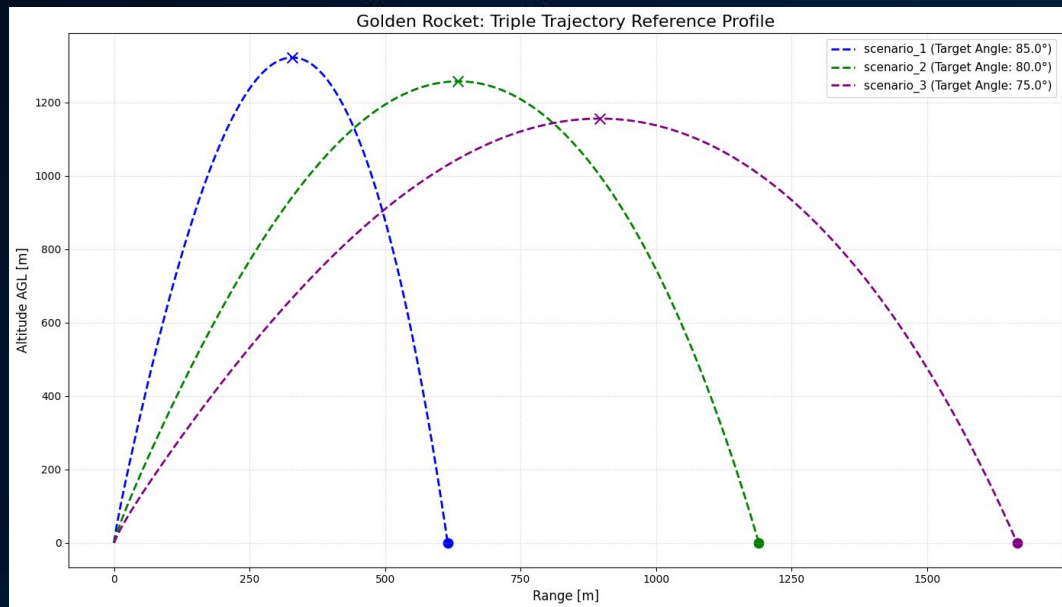
Fitness Function for Triple Trajectory Optimization

Scenario	Angle (θ)	Range (R_{ref})	Apogee (A_{ref})	TOF
High Altitude	85.0°	615.6 m	1322.3 m	35.6 s
Mid Range	80.0°	1189.9 m	1257.5 m	34.8 s
Long Range	75.0°	1666.5 m	1156.0 m	33.5 s

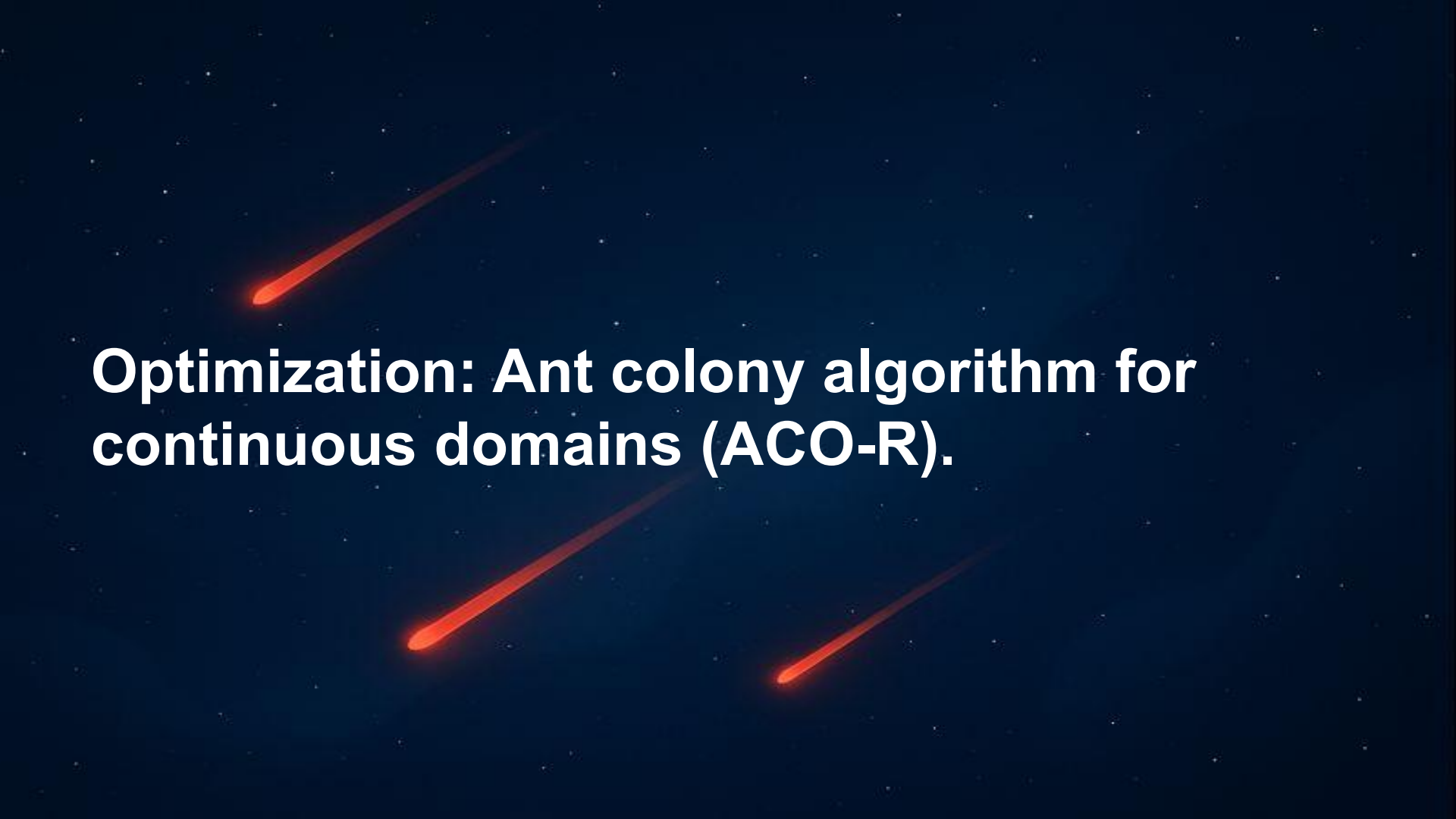
> **Note:** The Target Burn Time (TBO) remains constant at **4.50 s** for all scenarios, representing the intrinsic property of the reference motor.

$$Fitness = Fitness_{85^\circ} + Fitness_{80^\circ} + Fitness_{75^\circ}$$

Trajectory Visualization

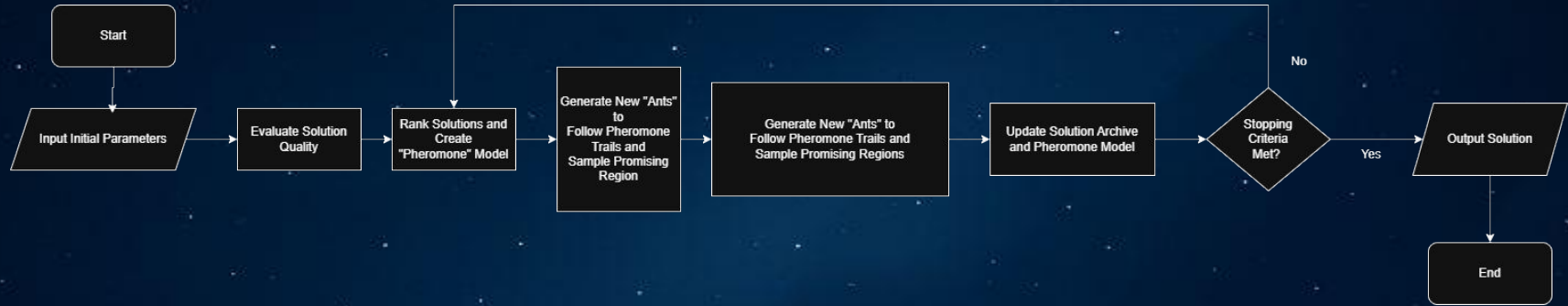


- This plot visualizes the **Ground Truth** performance generated by our reference rocket across three specific launch inclinations: 85°, 80°, and 75°.
- Note that **Task A (Single Trajectory Optimization)** focuses **exclusively on the Blue profile (85°)**.
- The Triple Trajectory task will require a single design to match all three curves simultaneously.



Optimization: Ant colony algorithm for continuous domains (ACO-R).

ACO-R: Pipeline



From discrete Graphs to Continuous Flight Physics

- Traditional Ant Colony Optimization (ACO) is designed for discrete problems (like TSP). To apply this metaheuristic to aerospace design, **ACO-R** an adaptation specifically developed for continuous search spaces (R^n) is used.
- **The Pheromone Analogy:** In a continuous domain, discrete pheromone trails do not exist. Instead, ACO-R replaces the pheromone matrix with a **Solution Archive** containing the k best results found so far.
- **Gaussian Kernel Sampling:** New candidate solutions *ants* are generated by sampling from a **Gaussian Probability Density Function (PDF)** constructed around the solutions in the archive. Better solutions in the archive have a higher *weight*, making it more likely for new ants to search in their neighbourhood (**Attractivity**).

Socha, K., & Dorigo, M. (2008). *Ant colony optimization for continuous domains*. European Journal of Operational Research.

ACO-R: Initialization

- The solution archive **T** is initialized with **k=50** candidate solutions, randomly generated within the specified bounds of the 11 design variables.
- Rank-based weights (w_j) are pre-computed using a Gaussian function with an elitism parameter of **q=0.1**, enforcing a highly greedy search strategy. Each solution in the initial archive is then evaluated via flight simulation to determine its fitness.
- Finally, the archive is sorted in ascending order of fitness cost, and the pre-computed weights are mapped to the solutions according to their rank.

$$w_j = \frac{1}{qk\sqrt{2\pi}} \exp \frac{-(rank(j)) - 1}{k - 1}$$

ACO-R: Optimization step

- The optimization process iterates until the computational budget of **4,000** function evaluations is exhausted. In each iteration **m=2** new candidate solutions "ants" are generated. The construction of a new solution vector depends on the selection of a guide solution ($\mathbf{S}_{\text{guide}}$) from the archive, which determines the center of the search space.
- The guide selection mechanism incorporates an *extra-elitism* probability parameter, $\mathbf{q}_{\text{mod}}=0.30$. For each new ant, the algorithm selects the best-ranked solution in the archive with probability $\mathbf{q}_{\text{mod}}$. Otherwise (with probability $1-\mathbf{q}_{\text{mod}}$), the guide is selected via *roulette wheel selection* based on the pre-computed rank weights $\mathbf{w}_j$.

ACO-R: Optimization step

- Once a guide $\mathbf{S}_{\text{guide}}$ is selected, its configuration vector serves as the mean vector $\boldsymbol{\mu}=[\theta_1, \dots, \theta_{11}]$. A standard deviation vector $\boldsymbol{\sigma}$ is derived based on the average distance between the guide and the other solutions in the archive, scaled by the convergence speed parameter ζ (*pheromone evaporation rate*).
- The new candidate parameters are then sampled from the resulting Gaussian Probability Density Function: $\mathbf{S}_{\text{new}} \sim \mathbf{N}(\boldsymbol{\mu}, \boldsymbol{\sigma})$
- To ensure physical feasibility, boundary clipping is applied immediately after sampling: if a generated parameter θ_i exceeds the defined design space $[LB_i, UB_i]$, it is clamped to the nearest boundary. Finally, the flight simulation is executed using $\mathbf{S}_{\text{new}}$ to compute its fitness value.

$$\sigma_j^i = \zeta \sum_{r=1}^k \frac{\text{abs}(s_i^r - s_i^j)}{k - 1}$$

ACO-R: Pheromone update & Mini-cycles

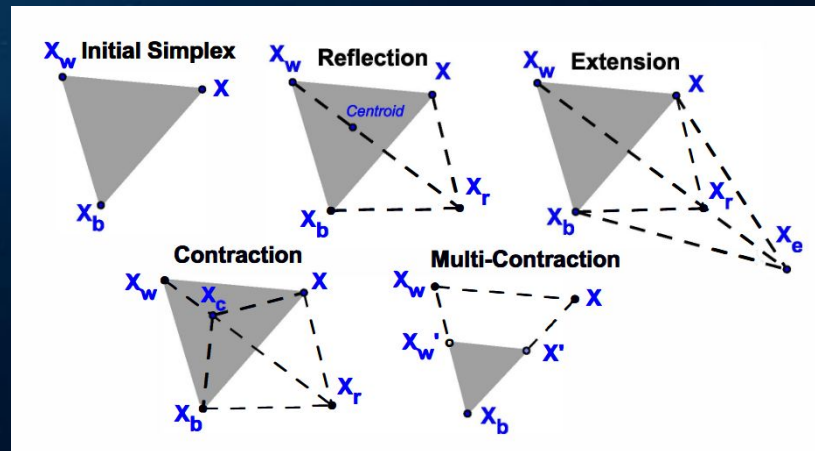
- At the end of each iteration, the expanded archive is sorted based on fitness values, and the worst-performing candidates are **pruned** to keep only the top k solutions.
- To prevent stagnation, the algorithm employs a **Mini-cycle strategy**: if 50 consecutive iterations pass without improvement, the archive is reset by preserving only the single best solution and populating the remaining $k-1$.



Optimization: Implementation of Nelder-Mead Local search

Local search: Strategy

- A derivative-free **direct search method** used to refine the rough solutions found by the global optimizer. It is particularly effective for cost functions where calculating gradients is computationally expensive or impossible.
- The algorithm iteratively evolves the simplex geometry to navigate the objective function landscape towards a **local minimum**, employing four distinct topological transformations:
 - Reflection
 - Expansion
 - Contraction
 - Multi-Contraction



Local search: Implementation

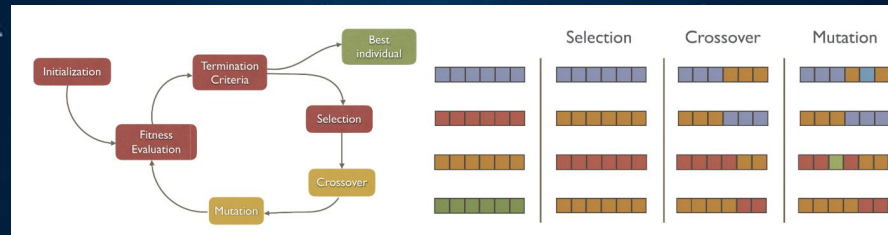
- This implementation employs a **Sequential Hybrid approach** to solve the fine-tuning problem, using the Nelder-Mead simplex to refine the rough solution provided by the ACO-R algorithm.
- The process begins by constructing an initial simplex of **$N+1$** vectors around the best solution found so far, applying a 5% step size perturbation to explore the immediate vicinity.
- Operating within a budget of **500 function calls**, the algorithm iteratively improves the solution by **reflecting** the worst point through the centroid; if this yields a new best, the simplex **expands** to stretch further in that direction, whereas a poor result triggers a **contraction** or a **multi-contraction** of all points around the best vertex.
- Finally, the algorithm sorts the resulting vectors and returns the optimized parameters associated with the lowest calculated fitness.



Optimization: Binary Genetic Algorithm (GA).

Benchmarking Algorithm: Binary GA

- **Binary Genetic Algorithm (GA)** is used as the comparative baseline for the ACO-R implementation, representing a classical evolutionary approach.
- Unlike the ACO-R, which operates in a continuous domain, this algorithm treats the optimization problem as a combinatorial challenge by **discretizing** the search space.
- Each continuous design variable (e.g. Radius, Impulse) is encoded into a **16-bit binary string**, concatenating them to form a single "chromosome" representing the rocket.



Binary GA implementation

- The algorithm evolves a population of solutions through **Tournament Selection** ($k=3$) to choose parents, **Single-Point Crossover** ($p_{cross} = 0.8$) to recombine genetic traits, and **Bit-Flip Mutation** ($p_{mut} = 0.02$) to introduce random diversity.
- **Phenotype Decoding**: Before every simulation, the binary strings are decoded back into floating-point values within the defined physical bounds to evaluate the fitness function.
- Finally, the system initializes a population of size $N=50$ with **Elitism** enabled to preserve the best solutions. The algorithm evolves this population until it reaches the stop condition of 4,000 fitness function calls.

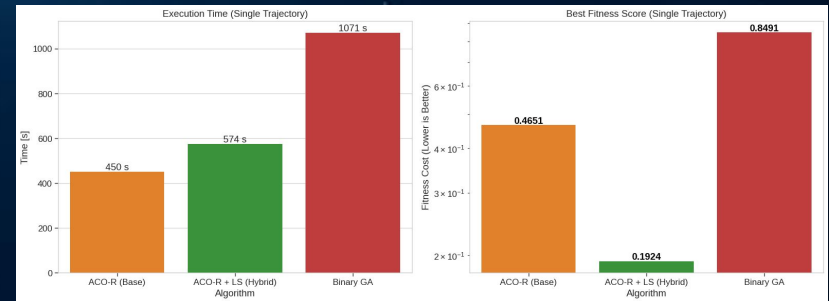


Experimental Results: Single Trajectory Optimization

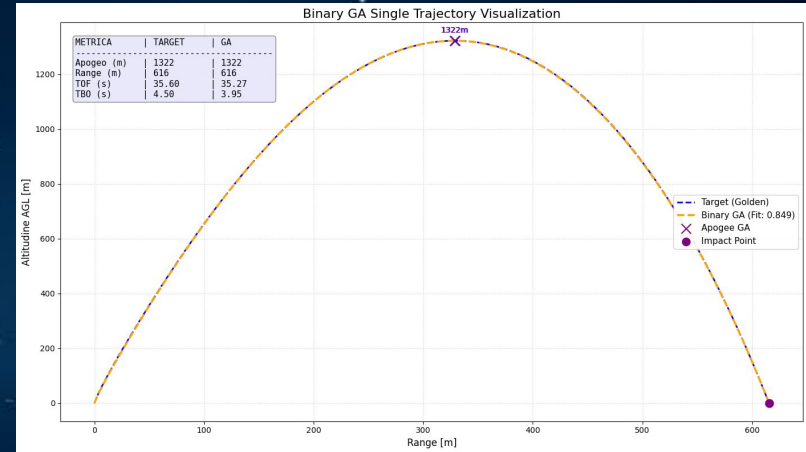
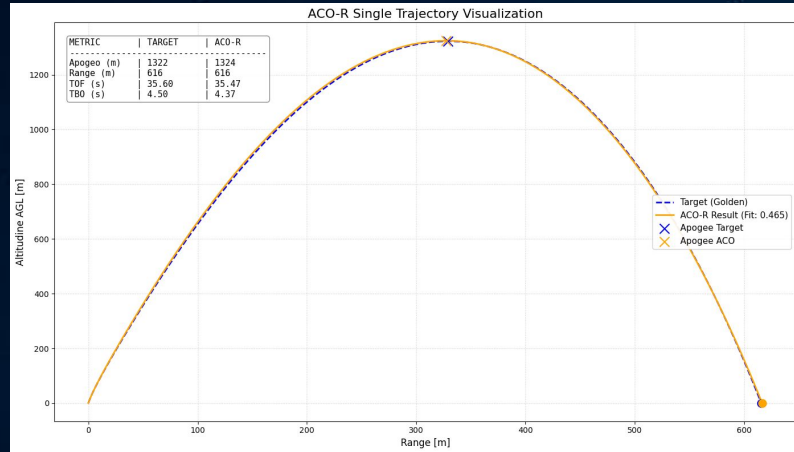
Algorithms Performance for the Single Trajectory

Algorithm	Exec Time (s)	Best Fitness	Range Err (m)	Apogee Err (m)	TOF Err (s)	TBO Err (s)
ACO-R (Base)	449.6031	0.4651	0.8018	1.7305	0.1284	0.1335
ACO-R + LS (Hybrid)	573.9021	0.1924	0.0002	0.0003	0.1344	0.1080
Binary GA	1070.8993	0.8491	0.0765	0.1391	0.3306	0.5469

The **Hybrid ACO-R + Local Search** strategy proved a better optimal solution, achieving near-perfect trajectory matching with a fitness of **0.1924** while requiring approximately half the computational time of the **Binary Genetic Algorithm** benchmark.



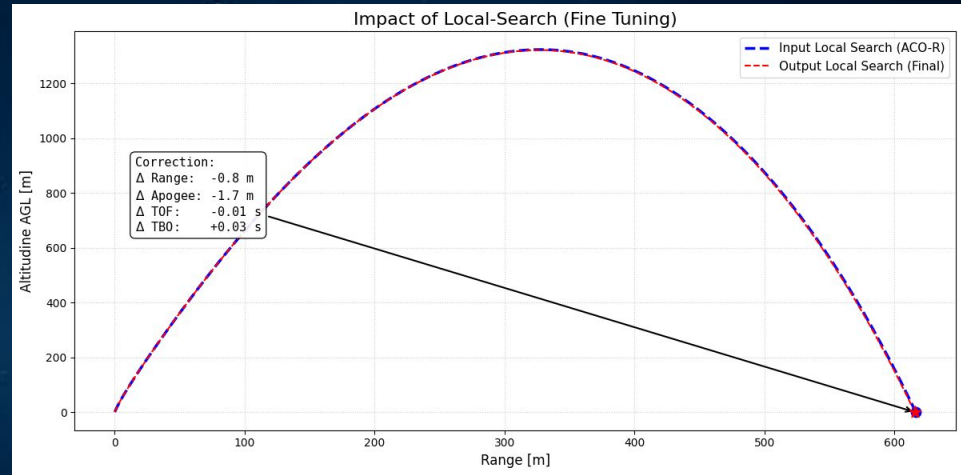
Base Optimization Algorithms: Single Trajectory Optimization



Baseline performance of the two global search algorithms on the Single Trajectory task. On the left, the **ACO-R** achieves a fitness of **0.465**, while on the right, the **Binary GA** reaches a fitness of **0.849**. Both algorithms successfully generate a trajectory that visually overlaps the target (blue dashed line),

Impact of The Local Search in the Single Trajectory Scenario

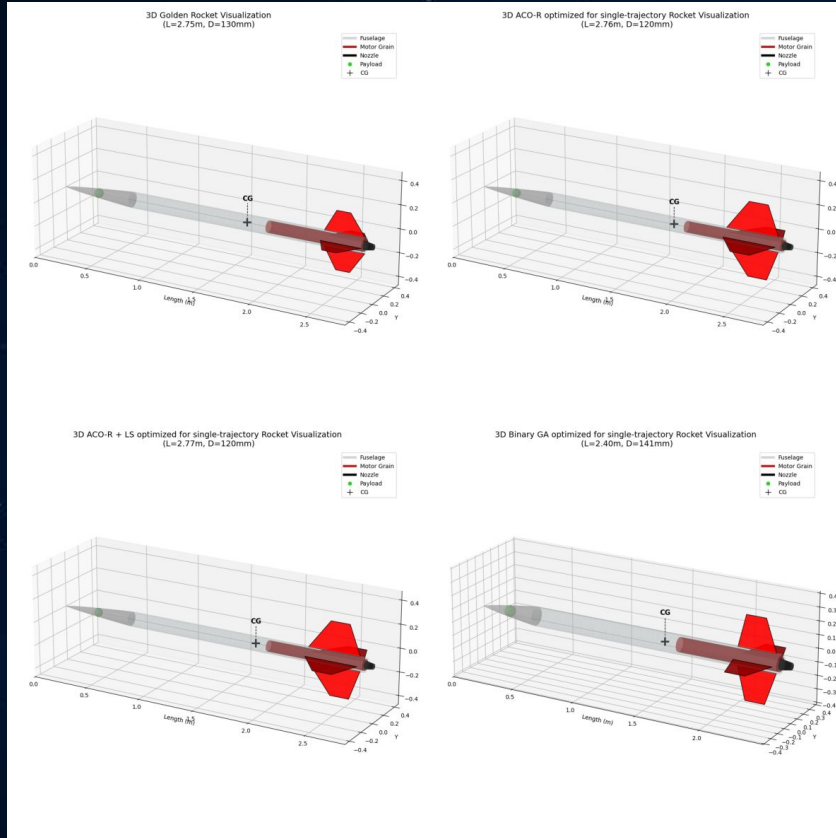
The Local Search fine-tuning phase applied minor adjustments to the temporal metrics (**TOF** and **TBO**) while applying a higher correction to the geometric errors in **Range** and **Apogee**. This refinement reduced the fitness cost from **0.4651** to **0.1924**, resulting in a substantial **58.63%** overall improvement.



Parameters Evolution: Single Trajectory Optimization

Parameter	Target (Golden)	Random Start (ACO-R)	Optimized (ACO-R)	ACO-R + LS	Random Start (GA)	Optimized (GA)
radius	0.065	0.0634	0.0601	0.0601	0.0725	0.0703
body_length	2.1	2.384	2.125	2.1257	1.5357	1.9495
nose_length	0.65	0.5497	0.6371	0.6441	0.6987	0.4516
fin_span	0.18	0.1556	0.2491	0.249	0.1934	0.2422
fin_root_chord	0.35	0.4438	0.4502	0.4503	0.2483	0.2943
fin_tip_chord	0.12	0.2394	0.1638	0.1643	0.0512	0.1505
fin_sweep_angle	30	44.51	37.5986	37.7994	39.583	14.0658
fin_distance_from	0	0.0377	0.0464	0.0468	0.0332	0.0336
total_impulse	3200	3128.7788	3141.1985	3142.6857	2631.4488	3324.6052
burn_time	4.5	2.334	4.3665	4.392	2.1182	3.9531
launch_angle	85	85.1029	84.9825	85.0111	80.6683	84.6795

Rocket Design for Single Trajectory Optimization



- This visualization displays the physical geometry of the optimized designs alongside the reference Rocket.
- The **ACO-R** algorithms successfully converged to a similar configuration that mirrors the ground truth almost exactly.
- In contrast, the **Binary GA** identified a different design, resulting in a shorter and wider missile that likely compensates for its increased air resistance with a modified propulsion profile.

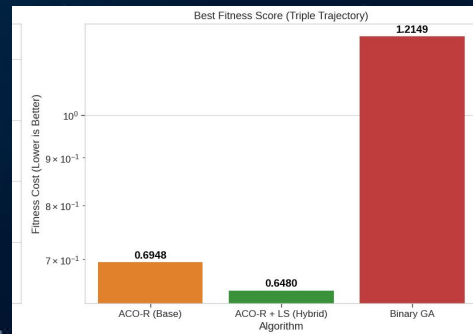


Experimental Results: Triple Trajectory Optimization

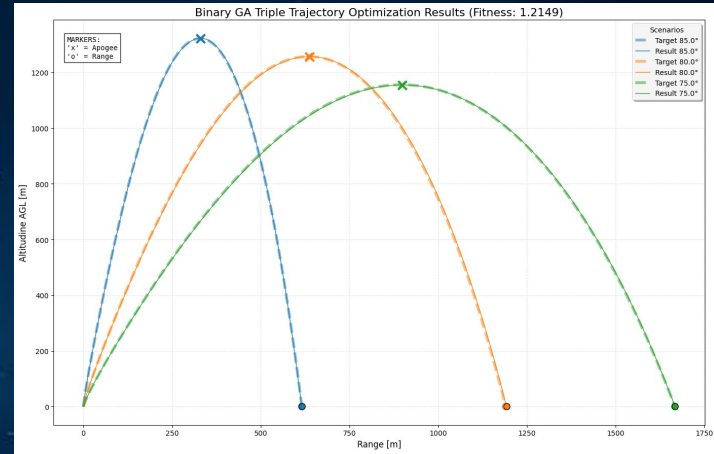
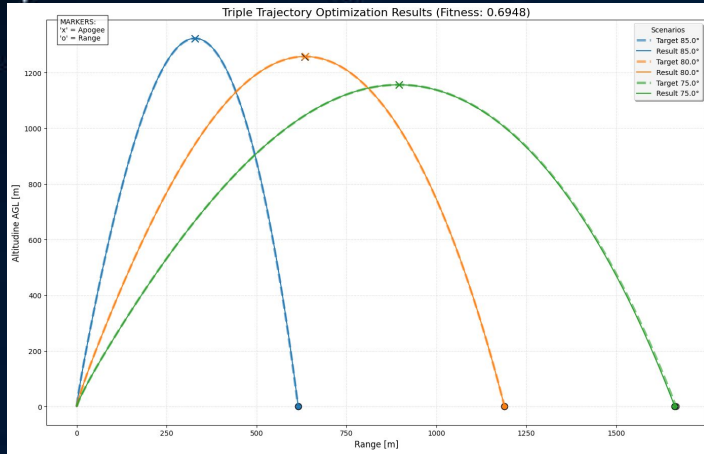
Algorithms Performance for the Triple Trajectory

Algorithm	Best Fitness	Exec Time (s)	Range Err (m)	Apogee Err (m)	TQF Err (s)	TBO Err (s)
ACO-R (Base)	0.6948	1594.5636	1.3211	0.2872	0.0518	0.0510
ACO-R + LS (Hybrid)	0.6480	1864.9235	1.2180	0.2086	0.0527	0.0494
Binary GA	1.2149	58109.9944	1.164	0.4849	0.0802	0.1254

Again, the **Hybrid ACO-R + LS** method halved the computational cost of the benchmark **GA** while achieving a superior fitness of **0.6480**, resulting in near-perfect triple trajectory reconstruction.



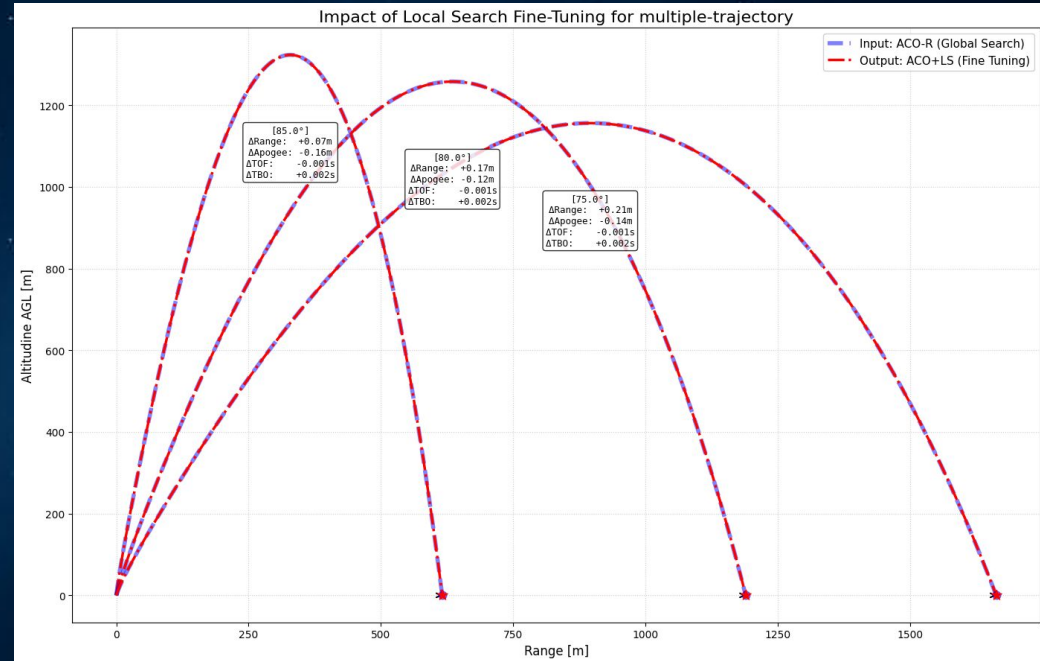
Base Optimization Algorithms: Triple Trajectory Optimization



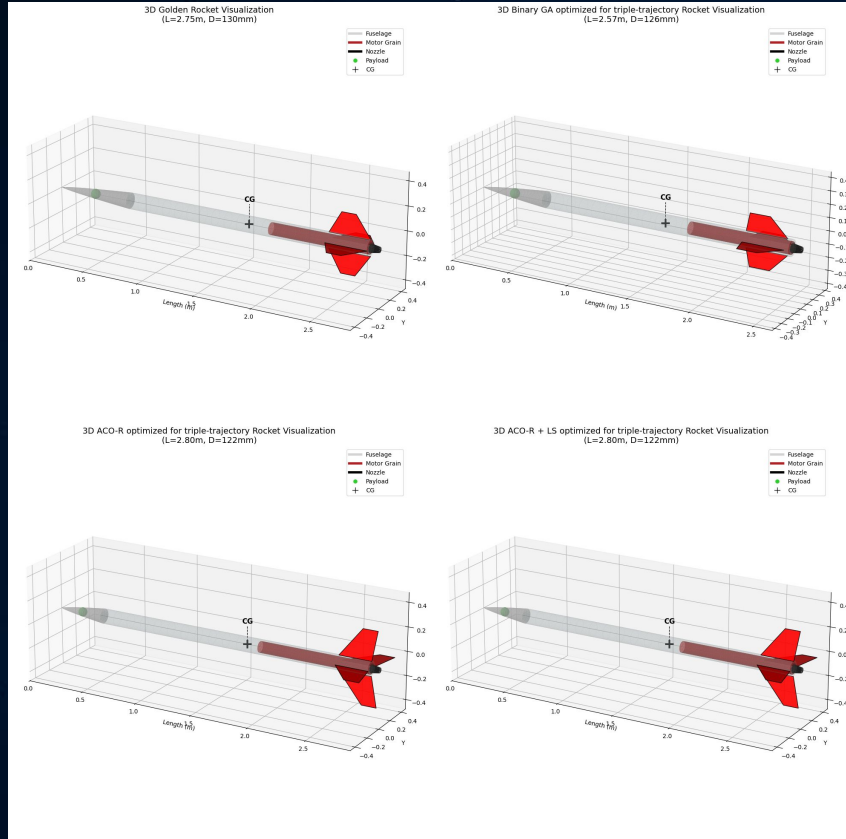
In the Triple Trajectory scenario, the **ACO-R** algorithm (left graph) proved more effective at resolving the physical trade-offs required to match three different flight paths with a single vehicle design. Despite visually similar trajectory plots, the fitness metrics shows that the continuous search strategy achieved nearly double the accuracy of the **Binary GA**.

Impact of The Local Search in the Triple Trajectory Scenario

- **Fine-tuning capability** of the Hybrid approach: while the visual overlap between the **Input (ACO-R)** and **Output (ACO+LS)**, the local search strategy performed **micro-corrections**.
- Nelder-Mead algorithm effectively resolved the residual stochastic errors left by the ACO, pushing the solution from "near-optimal" to high-precision convergence across all three flight scenarios.



Rocket Design for Triple Trajectory Optimization



- **Binary GA (Top Right):** Converged on a **shorter configuration** (L=2.57m), compensating for aerodynamic properties with a different mass distribution.
- **ACO-R & Hybrid (Bottom):** Both Continuous Ant Colony variants converged on a **longer, more slender design** (L=2.80m, D=122mm), favoring a higher fineness ratio to reduce pressure drag.

Parameters Evolution: Triple Trajectory Optimization

Parameter	Target (Golden)	Random Start (ACO-R)	Optimized (ACO-R)	ACO-R + LS	Optimized (GA)
radius	0.065	0.0717	0.0608	0.0608	0.0631
body_length	2.1	1.5787	2.3904	2.3904	2.0117
nose_length	0.65	0.7474	0.4118	0.4114	0.5577
fin_span	0.18	0.2378	0.2488	0.2488	0.1381
fin_root_chord	0.35	0.3016	0.278	0.2781	0.3246
fin_tip_chord	0.12	0.0686	0.1256	0.1256	0.1588
fin_sweep_angle	30	28.9184	38.6536	38.8959	11.5942
fin_distance_from_base	0	0.0482	0	0.0003	0.0369
total_impulse	3200	2966.5081	3253.8251	3253.6587	2905.333
burn_time	4.5	4.5701	4.449	4.4506	4.6254



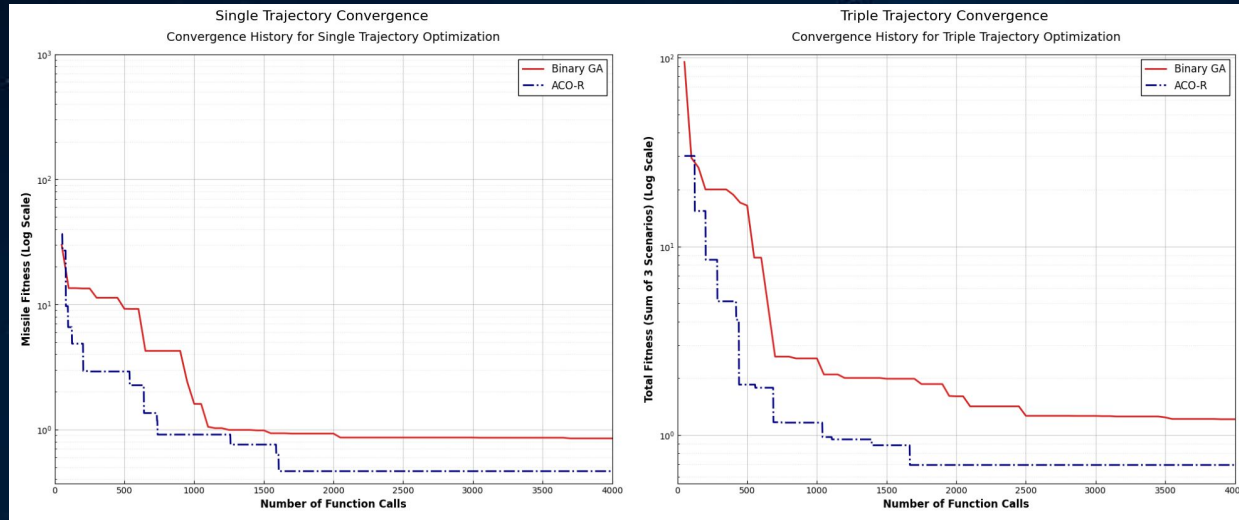
Experimental Results: Final Comparison

Fitness Results

Algorithm	Single Trajectory	Triple Trajectory
ACO-R (Base)	0.46511	0.6948
ACO-R + LS (Hybrid)	0.1924	0.6480
Binary GA	0.8491	1.2149

- The experimental results establishes lower fitness result of the **Hybrid ACO-R** strategy over the discrete evolutionary benchmark across both optimization tasks.
- In the **Single Trajectory** scenario, the integration of the **Local Search** proved to be decisive, demonstrating the necessity of fine-tuning for achieving precision.
- The **Triple Trajectory** challenge reveals the robustness of the continuous search domain, where both **ACO-R** variants significantly outperformed the Binary Genetic Algorithm, which struggled to resolve the complex physical trade-offs.

Convergence History



- **ACO-R** drastically reduces the fitness error within the first 1,000 function calls compared to the slower, step-wise progress of the Binary **Genetic Algorithm**.
- Across both single and triple trajectory scenarios, **ACO-R** consistently achieves a lower final fitness score, indicating a more precise aerodynamic solution.

Physical Error Comparison

- The **Single Trajectory** plot highlights the impact of the Local Search, where the Hybrid approach (green bars) reduces Range and Apogee errors by orders of magnitude compared to the baseline algorithms.
- In contrast, the **Triple Trajectory** results show higher errors due to the physical trade-offs required to satisfy multiple flight profiles.





Conclusions & Future Work

Conclusions

- This project achieved its primary objective of solving the inverse design problem for a high-power rocket, replicating complex flight trajectories with a good precision using a custom Python framework.
- The experimental results confirmed the best results of the **Hybrid ACO-R** approach over the discrete **Binary Genetic Algorithm** benchmark, demonstrating that combining global stochastic search with deterministic local fine-tuning is crucial for resolving the sensitivity of aerodynamic parameters in continuous domains.
- These findings align closely with the conclusions of *Kiyak et al.*, validating their hypothesis that **ACO-R** offers a more efficient convergence profile for ballistic optimization than traditional evolutionary methods.

Future Work

- Develop a **high-fidelity 6-DOF missile simulation** with stricter physical constraints and a more advanced rocket propulsion model.
- Introduce **Local Search (LS) refinement** at the end of each **mini-cycle** to improve solution quality (paper fidelity).
- Implement **mini-cycles for the binary Genetic Algorithm (GA)** as well, ensuring a **fair and consistent comparison** across optimization methods.
- Redesign the **fitness function as a multi-objective formulation**, while maintaining conceptual continuity with the objective function defined in the original paper.

References

- Kiyak, Z. J., Hartfield, R. J., & Ledlow, T. W. (2014). Missile Trajectory Optimization Using a Modified Ant Colony Algorithm. Auburn University.
<https://www.semanticscholar.org/paper/Missile-trajectory-optimization-using-a-modified-Kiyak-Hartfield/4b52688ff709a352cb08fc3c28126953b58ea21b>
- Socha, K., & Dorigo, M. (2008). Ant colony optimization for continuous domains. European Journal of Operational Research.
https://iridia.ulb.ac.be/~mdorigo/Published_papers/All_Dorigo_papers/SocDor2008ejor.pdf
- RocketPy Team. RocketPy: A Six-Degree-of-Freedom Rocket Trajectory Simulator.
<https://github.com/RocketPy-Team/RocketPy>

The background is a dark blue gradient filled with numerous small white dots, representing a starry night sky. Three bright red streaks, resembling shooting stars or meteors, are positioned diagonally across the frame. One streak is in the upper left, another in the lower left, and a third in the lower right. The text "Thanks for Listening!!" is centered in the middle of the image in a white, bold, sans-serif font.

Thanks for Listening!!